МЕТОДИЧКА ДЛЯ СТУДЕНТОВ

JavaScript: обычные функции, стрелочные функции, массивы и методы массивов

Для кого этот материал

Материал рассчитан на студентов, которые уже прошли основы JavaScript: переменные, типы данных, условия и циклы. В этой методичке мы спокойно и последовательно разбираем функции и массивы без объектов и без лишней сложности.

Что вы должны уметь после изучения темы

- объяснять, зачем нужны функции и когда их использовать;
- писать обычные функции и стрелочные функции;
- понимать разницу между параметрами, аргументами, console.log() и return;
- создавать массивы, обращаться к элементам по индексу и изменять их;
- перебирать массив с помощью цикла for;
- использовать базовые методы массива: push, pop, shift, unshift, includes, indexOf, join, slice;
- применять map, filter, find и forEach на простых массивах чисел и строк.

Как читать методичку

Сначала изучите теорию и внимательно разберите все примеры. После этого выполните практические задания в конце документа. Не пропускайте разбор return и цикл for по массиву: без этого методы массива будут путать.

1. Зачем вообще нужны функции

Когда программа становится длиннее, в ней очень быстро появляется повторяющийся код. Повторение - это одна из главных причин ошибок: вы исправили логику в одном месте, но забыли исправить в другом. Функция позволяет один раз описать нужное действие и потом вызывать его столько раз, сколько нужно.

Посмотрите на пример ниже. Код рабочий, но неудобный: одна и та же операция повторяется несколько раз.

```
let num1 = 5;
```

```
let num2 = 10;
let num3 = 15;

console.log(num1 * 2);
console.log(num2 * 2);
console.log(num3 * 2);
```

Если чисел будет не 3, а 30 или 300, такой подход станет неудобным. Нам нужен способ описать действие один раз. Именно это и делает функция.

2. Обычная функция: синтаксис и вызов

Обычная функция в JavaScript записывается через ключевое слово function. Сначала мы ее объявляем, а потом вызываем.

```
function sayHello() {
  console.log("Hello");
}

sayHello();
```

Разберем по частям:

- function - ключевое слово, которое говорит JavaScript: сейчас будет объявлена функция.
- sayHello - имя функции. По имени мы потом обращаемся к функции.
- () - круглые скобки. В них позже можно передавать данные.
- {} - тело функции, то есть код, который выполнится при вызове.

Важно

Объявление функции не запускает ее автоматически. Функция начнет выполняться только после вызова: sayHello().

Одну и ту же функцию можно вызвать много раз:

```
function sayHello() {
  console.log("Hello");
}

sayHello();
sayHello();
```

```
sayHello();
```

3. Параметры и аргументы

Очень часто функция должна работать не с одним и тем же значением, а с разными. Для этого используются параметры.

```
function greet(name) {
  console.log("Привет, " + name);
}

greet("Алина");
greet("Дамир");
greet("Саша");
```

Здесь name - это параметр. Он объявлен внутри функции и получает значение при вызове. Конкретные значения "Алина", "Дамир" и "Саша" - это аргументы.

Термин	Что это такое	Пример
Параметр	Переменная в объявлении функции	function greet(name) { ... }
Аргумент	Конкретное значение при вызове функции	greet("Алина")

Примеры функций с параметрами:

```
function sum(a, b) {
  console.log(a + b);
}

sum(2, 3);
sum(10, 5);
```

```
function multiply(a, b) {
  console.log(a * b);
}

multiply(4, 5);
multiply(7, 8);
```

```
function checkAge(age) {
  if (age >= 18) {
    console.log("Доступ разрешен");
  } else {
    console.log("Доступ запрещен");
  }
}

checkAge(20);
checkAge(15);
```

```
function shout(word) {
  console.log(word.toUpperCase());
}

shout("hello");
shout("javascript");
```

4. Console.log() и return - это не одно и то же

Это одна из самых важных тем. Студенты очень часто путают вывод результата на экран и возврат результата из функции.

Что делает	Когда использовать	Пример
console.log()	Просто выводит значение на экран	console.log(5);
return	Возвращает значение из функции наружу	return 5;

Сначала пример без return:

```
function sum(a, b) {
  console.log(a + b);
}

sum(2, 3);
```

Такой код выведет 5 в консоль. Но если вы попробуете сохранить результат в переменную, получится проблема:

```
function sum(a, b) {
  console.log(a + b);
}

let result = sum(2, 3);
console.log(result);
```

В переменной result окажется undefined, потому что функция ничего не вернула. Она только вывела значение на экран.

Теперь правильный вариант:

```
function sum(a, b) {
  return a + b;
}

let result = sum(2, 3);
console.log(result);
```

Теперь функция действительно возвращает значение, и его можно сохранить, передать дальше или использовать в другом выражении.

Ключевая мысль

console.log() нужен, чтобы показать значение человеку. return нужен, чтобы передать значение программе дальше.

Пошаговый разбор этого примера:

- Вызов sum(2, 3) передает в функцию два аргумента: 2 и 3.
- Параметры a и b получают значения 2 и 3.
- Внутри функции вычисляется a + b.
- return возвращает результат наружу.
- Этот результат попадает в переменную result.

Еще один очень важный факт: как только срабатывает return, функция завершает работу. Все, что идет после него, уже не выполняется.

```
function test() {
  console.log("Начало");
```

```
  return 10;
  console.log("Конец");
}

console.log(test());
```

5. Подробные примеры обычных функций

Ниже идет большая серия простых и понятных примеров. Разбирайте их по одному и после каждого пробуйте менять аргументы.

Удвоить число

```
function double(num) {
  return num * 2;
}

console.log(double(5));
console.log(double(12));
```

Функция получает число num и возвращает результат умножения на 2.

Найти квадрат числа

```
function square(num) {
  return num * num;
}

console.log(square(4));
console.log(square(9));
```

Один и тот же параметр участвует в умножении два раза.

Сумма двух чисел

```
function add(a, b) {
  return a + b;
}

console.log(add(2, 8));
console.log(add(100, 50));
```

Функция получает два числа и складывает их.

Разница двух чисел

```
function subtract(a, b) {
  return a - b;
}

console.log(subtract(10, 3));
console.log(subtract(50, 20));
```

Из первого числа вычитается второе.

Проверка на четность

```
function isEven(num) {
  return num % 2 === 0;
}

console.log(isEven(4));
console.log(isEven(7));
```

Оператор % вычисляет остаток от деления. Если остаток 0, число четное.

Проверка возраста

```
function isAdult(age) {
  return age >= 18;
}

console.log(isAdult(20));
console.log(isAdult(16));
```

Функция возвращает логическое значение true или false.

Длина строки

```
function getLength(text) {
  return text.length;
}

console.log(getLength("hello"));
console.log(getLength("javascript"));
```

У строки есть свойство length - количество символов.

Верхний регистр

```
function toUpper(text) {
  return text.toUpperCase();
}

console.log(toUpper("hello"));
console.log(toUpper("frontend"));
```

Метод toUpperCase() переводит строку в верхний регистр.

Добавить знак

```
function makeLoud(text) {
  return text + "!";
}

console.log(makeLoud("Привет"));
console.log(makeLoud("JavaScript"));
```

Функция склеивает исходную строку и восклицательный знак.

Цена со скидкой 10%

```
function applyDiscount(price) {
  return price * 0.9;
}

console.log(applyDiscount(1000));
console.log(applyDiscount(2500));
```

Если нужно оставить 90% от цены, достаточно умножить цену на 0.9.

Большее из двух чисел

```
function getMax(a, b) {
  if (a > b) {
    return a;
  } else {
    return b;
  }
}
```

```
console.log(getMax(10, 5));
console.log(getMax(3, 8));
```

Через if/else можно вернуть одно из двух значений в зависимости от условия.

6. Стрелочные функции

Стрелочная функция - это более короткая запись функции. На этом этапе главное понять синтаксис. Углубляться в тему this пока не нужно.

Сначала вспомним обычную запись:

```
function square(num) {
  return num * num;
}
```

Та же самая логика в стрелочном виде:

```
const square = (num) => {
  return num * num;
};
```

Если в функции одна строка и она сразу возвращает результат, запись можно сократить:

```
const square = num => num * num;
```

Вид функции	Запись	Комментарий
Обычная функция	function square(num) { return num * num; }	Более привычный и длинный синтаксис
Стрелочная функция	const square = num => num * num;	Короткая запись той же логики

Ниже - несколько подробных примеров стрелочных функций.

```
const add = (a, b) => a + b;
console.log(add(2, 3));
```

```
const double = num => num * 2;
console.log(double(7));
```

```
const shout = word => word.toUpperCase();
console.log(shout("hello"));
```

```
const isEven = num => num % 2 === 0;
console.log(isEven(6));
console.log(isEven(9));
```

```
const isAdult = age => age >= 18;
console.log(isAdult(22));
console.log(isAdult(15));
```

```
const getLength = text => text.length;
console.log(getLength("JavaScript"));
```

```
const applyDiscount = price => price * 0.9;
console.log(applyDiscount(5000));
```

Вывод по теме

По смыслу обычные и стрелочные функции делают одно и то же. На базовом уровне разница только в записи. Обычные функции нужно понимать обязательно, потому что стрелочная запись опирается на эти же идеи: параметры, аргументы и return.

7. Типичные ошибки при работе с функциями

- Студент объявил функцию, но не вызвал ее.
- Студент пишет console.log() внутри функции и думает, что значение вернулось наружу.
- Студент забывает return и потом не понимает, почему переменная стала undefined.
- Студент путает параметр и аргумент.
- Студент пытается использовать имя переменной, которой внутри функции нет.

Как себя проверять

После написания функции спросите себя: что она принимает на вход, что делает внутри и что возвращает на выход. Если на выходе должно быть значение, почти всегда нужен return.

8. Что такое массив

После функций мы переходим к массивам. Массив - это список значений, хранящийся в одной переменной. Это очень удобно, когда данных много.

Сравните два подхода:

```
let student1 = "Алина";
let student2 = "Дамир";
let student3 = "Саша";
let student4 = "Ильяс";
```

Так хранить данные можно, но неудобно. Намного лучше собрать их в один массив:

```
let students = ["Алина", "Дамир", "Саша", "Ильяс"];
```

В массиве значения записываются в квадратных скобках, а элементы отделяются запятыми.

Примеры массивов:

```
let numbers = [10, 20, 30, 40, 50];
let fruits = ["apple", "banana", "orange"];
let words = ["hi", "hello", "javascript"];
```

9. Индексы массива

Каждый элемент массива имеет свой индекс. Индексация начинается с 0, а не с 1.

```
let fruits = ["apple", "banana", "orange"];

console.log(fruits[0]);
console.log(fruits[1]);
console.log(fruits[2]);
```

- fruits[0] - первый элемент массива.
- fruits[1] - второй элемент массива.
- fruits[2] - третий элемент массива.

Элемент по индексу можно не только прочитать, но и изменить:

```
let fruits = ["apple", "banana", "orange"];
fruits[1] = "mango";

console.log(fruits);
```

Важно

Если студент забывает, что индексация начинается с нуля, он почти всегда получает не тот элемент, который ожидал.

10. Длина массива: length

Свойство length показывает, сколько элементов находится в массиве.

```
let fruits = ["apple", "banana", "orange"];
console.log(fruits.length);
```

Если в массиве три элемента, fruits.length вернет 3.

11. Перебор массива через for

Пока вы не научились уверенно проходить массив через обычный цикл for, методы массива будут казаться магией. Поэтому этот блок надо понять особенно хорошо.

```
let numbers = [10, 20, 30, 40];

for (let i = 0; i < numbers.length; i++) {
  console.log(numbers[i]);
}
```

Разберем цикл по шагам:

- let i = 0 - начинаем с нулевого индекса, то есть с первого элемента массива.
- i < numbers.length - продолжаем цикл, пока индекс меньше длины массива.
- i++ - после каждой итерации переходим к следующему индексу.
- numbers[i] - берем текущий элемент массива.

Несколько полезных примеров на цикл по массиву:

Вывести все элементы массива

```
let fruits = ["apple", "banana", "orange"];

for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}
```

Вывести каждое слово в верхнем регистре

```
let words = ["hello", "javascript", "array"];

for (let i = 0; i < words.length; i++) {
  console.log(words[i].toUpperCase());
}
```

Умножить все числа на 2

```
let nums = [5, 10, 15, 20];

for (let i = 0; i < nums.length; i++) {
  console.log(nums[i] * 2);
}
```

Найти сумму всех чисел

```
let nums = [10, 20, 30, 40];
let sum = 0;

for (let i = 0; i < nums.length; i++) {
  sum = sum + nums[i];
}

console.log(sum);
```

Здесь переменная sum сначала равна 0, а потом в нее постепенно добавляется каждый элемент массива.

Найти максимальное число

```
let nums = [8, 3, 15, 2, 10];
let max = nums[0];
```

```
for (let i = 1; i < nums.length; i++) {
  if (nums[i] > max) {
    max = nums[i];
  }
}

console.log(max);
```

Почему цикл начинается с 1? Потому что max уже получил значение первого элемента nums[0], и повторно сравнивать его с самим собой не нужно.

Посчитать количество четных чисел

```
let nums = [1, 2, 3, 4, 5, 6];
let count = 0;

for (let i = 0; i < nums.length; i++) {
  if (nums[i] % 2 === 0) {
    count++;
  }
}

console.log(count);
```

12. Базовые методы массива

Ниже идут методы, которые используются очень часто. Их нужно знать уверенно.

push() - добавить элемент в конец массива

```
let fruits = ["apple", "banana"];
fruits.push("orange");

console.log(fruits);
```

После выполнения к массиву добавится новый элемент в конец.

pop() - удалить последний элемент

```
let fruits = ["apple", "banana", "orange"];
fruits.pop();

console.log(fruits);
```

shift() - удалить первый элемент

```
let fruits = ["apple", "banana", "orange"];
fruits.shift();

console.log(fruits);
```

unshift() - добавить элемент в начало

```
let fruits = ["banana", "orange"];
fruits.unshift("apple");

console.log(fruits);
```

includes() - проверить, есть ли элемент в массиве

```
let fruits = ["apple", "banana", "orange"];

console.log(fruits.includes("banana"));
console.log(fruits.includes("mango"));
```

indexOf() - найти индекс элемента

```
let fruits = ["apple", "banana", "orange"];

console.log(fruits.indexOf("banana"));
console.log(fruits.indexOf("mango"));
```

join() - склеить массив в одну строку

```
let words = ["Я", "учу", "JavaScript"];
let sentence = words.join(" ");

console.log(sentence);
```

slice() - взять часть массива

```
let fruits = ["apple", "banana", "orange", "mango"];

console.log(fruits.slice(1, 3));
```

Запомните

slice() не меняет исходный массив. Он возвращает новый массив с выбранной частью элементов.

13. Метод map()

map() нужен, когда надо пройти по массиву и преобразовать каждый элемент. На выходе map() создает новый массив той же длины.

Сначала посмотрим, как такая задача решается через обычный цикл:

```
let nums = [1, 2, 3, 4];
let doubled = [];

for (let i = 0; i < nums.length; i++) {
  doubled.push(nums[i] * 2);
}

console.log(doubled);
```

То же самое через map():

```
let nums = [1, 2, 3, 4];
let doubled = nums.map(num => num * 2);

console.log(doubled);
```

Как это читать: для каждого элемента массива nums взять num и вернуть num * 2. Все результаты собрать в новый массив doubled.

Еще примеры map():

```
let words = ["apple", "banana", "orange"];
let upperWords = words.map(word => word.toUpperCase());

console.log(upperWords);
```

```
let words = ["hi", "hello", "javascript"];
let lengths = words.map(word => word.length);

console.log(lengths);
```

```
let words = ["hello", "wow", "nice"];
let loudWords = words.map(word => word + "!");

console.log(loudWords);
```

```
let prices = [1000, 2000, 3000];
let discountedPrices = prices.map(price => price * 0.9);

console.log(discountedPrices);
```

Главное правило map()

map() не изменяет старый массив автоматически. Он создает новый массив. Если вы хотите получить результат, его нужно сохранить в переменную.

14. Метод filter()

filter() нужен, когда надо оставить только те элементы, которые подходят под условие. На выходе получается новый массив, но его длина уже может быть меньше исходной.

Сначала решение через for:

```
let nums = [1, 2, 3, 4, 5, 6];
let evenNums = [];

for (let i = 0; i < nums.length; i++) {
  if (nums[i] % 2 === 0) {
    evenNums.push(nums[i]);
  }
}

console.log(evenNums);
```

Теперь то же самое через filter():

```
let nums = [1, 2, 3, 4, 5, 6];
let evenNums = nums.filter(num => num % 2 === 0);

console.log(evenNums);
```

Если условие возвращает true, элемент остается. Если false, элемент отбрасывается.

Еще примеры filter():

```
let nums = [4, 12, 7, 20, 3, 15];
let bigNums = nums.filter(num => num > 10);

console.log(bigNums);
```

```
let words = ["cat", "elephant", "dog", "javascript"];
let shortWords = words.filter(word => word.length <= 3);

console.log(shortWords);
```

```
let words = ["apple", "banana", "kiwi", "orange"];
let longWords = words.filter(word => word.length > 5);
```

```
console.log(longWords);
```

```
let words = ["apple", "banana", "avocado", "mango"];
let aWords = words.filter(word => word.startsWith("a"));

console.log(aWords);
```

15. Метод find()

find() похож на filter(), но возвращает не массив, а только первый элемент, который подошел под условие.

Метод	Что возвращает	Когда использовать
filter()	Новый массив из всех подходящих элементов	Когда нужно оставить все подходящие значения
find()	Первый подходящий элемент	Когда достаточно найти только одно первое совпадение

```
let nums = [3, 7, 12, 20, 25];
let result = nums.find(num => num > 10);

console.log(result);
```

Результатом будет 12, потому что это первое число больше 10.

```
let nums = [1, 3, 5, 8, 10];
let even = nums.find(num => num % 2 === 0);

console.log(even);
```

```
let words = ["hi", "cat", "banana", "dog"];
let word = words.find(item => item.length > 5);

console.log(word);
```

16. Метод forEach()

forEach() просто проходит по массиву и выполняет действие для каждого элемента. В отличие от map(), он не создает новый массив.

```
let fruits = ["apple", "banana", "orange"];

fruits.forEach(fruit => {
  console.log("Фрукт:", fruit);
});
```

```
let nums = [10, 20, 30];

nums.forEach(num => {
  console.log(num * 2);
});
```

Как запомнить разницу

Если нужно просто пройтись и что-то сделать с каждым элементом - часто подходит forEach(). Если нужен новый массив после преобразования - чаще нужен map().

17. Простые комбинации методов

Когда базовые методы уже понятны, их можно комбинировать. Главное - читать цепочку по шагам слева направо.

```
let nums = [1, 2, 3, 4, 5, 6];

let result = nums
  .filter(num => num % 2 === 0)
  .map(num => num * 10);

console.log(result);
```

Сначала filter() оставляет только четные числа: [2, 4, 6]. Затем map() умножает каждое из них на 10: [20, 40, 60].

```
let words = ["apple", "banana", "kiwi", "orange"];

let result = words
```

```
  .filter(word => word.length > 4)
  .map(word => word.toUpperCase());

console.log(result);
```

Здесь сначала остаются слова длиннее 4 букв, а потом они переводятся в верхний регистр.

18. Большой блок практики

Эти задания рассчитаны на самостоятельную работу после изучения темы. Сначала решайте обычным способом, а потом, где возможно, пробуйте записать решение короче.

Часть A. Обычные функции

- Напишите функцию double(num), которая возвращает число, умноженное на 2.
- Напишите функцию square(num), которая возвращает квадрат числа.
- Напишите функцию sum(a, b), которая возвращает сумму двух чисел.
- Напишите функцию isPositive(num), которая возвращает true, если число положительное.
- Напишите функцию isEven(num), которая возвращает true, если число четное.
- Напишите функцию getLength(text), которая возвращает длину строки.
- Напишите функцию toUpper(text), которая переводит строку в верхний регистр.
- Напишите функцию addDot(text), которая добавляет точку в конец строки.
- Напишите функцию getFirstChar(text), которая возвращает первый символ строки.
- Напишите функцию applyDiscount(price), которая уменьшает цену на 10%.

Часть B. Стрелочные функции

- Перепишите задания 1-5 из части A в стрелочном виде.
- Напишите стрелочную функцию getMax(a, b), которая возвращает большее из двух чисел.
- Напишите стрелочную функцию makeLoud(text), которая добавляет к строке восклицательный знак.

Часть C. Массивы и цикл for

```
let nums = [5, 12, 8, 20, 3, 15];
```

- Выведите все элементы массива nums.
- Выведите каждый элемент массива nums, умноженный на 2.
- Найдите сумму всех элементов массива nums.
- Найдите максимальный элемент массива nums.
- Посчитайте, сколько в массиве nums четных чисел.

Часть D. Методы массива

```
let words = ["apple", "hi", "banana", "cat", "orange"];
```

- Получите новый массив, где каждое слово переведено в верхний регистр.
- Получите новый массив, состоящий из длин всех слов.
- Оставьте только слова длиннее 4 букв.
- Найдите первое слово длиннее 5 букв.
- Проверьте, есть ли в массиве слово "banana".
- Склейте все слова в одну строку через пробел при помощи join().

Часть E. Дополнительные задания

- Создайте массив из 5 чисел и добавьте в конец еще одно число через push().
- Удалите последний элемент массива через pop().
- Добавьте новый элемент в начало массива через unshift().
- Удалите первый элемент массива через shift().
- Возьмите часть массива через slice().
- Создайте массив чисел и получите новый массив, где все числа больше 10 умножены на 3.

19. Домашнее задание

Сделайте все задания аккуратно, с понятными именами переменных. После каждой задачи проверяйте результат через console.log().

Блок 1. Функции

- Напишите функцию triple(num), которая утраивает число.
- Напишите функцию isNegative(num), которая проверяет, является ли число отрицательным.
- Напишите функцию lastLetter(text), которая возвращает последний символ строки.
- Напишите функцию makeQuestion(text), которая добавляет к строке знак вопроса.
- Перепишите все функции из этого блока в стрелочном виде.

Блок 2. Массив чисел

```
let nums = [7, 14, 2, 9, 20, 11, 4];
```

- Найдите сумму всех чисел.
- Найдите максимальный элемент.
- Оставьте только четные числа.
- Получите новый массив, где каждое число увеличено на 5.
- Найдите первое число больше 10.

Блок 3. Массив строк

```
let words = ["red", "blue", "green", "hi", "yellow", "cat"];
```

- Переведите все слова в верхний регистр.
- Получите массив длин слов.
- Оставьте только слова длиннее 3 букв.
- Найдите первое слово, начинающееся с буквы "g".
- Склейте слова через запятую.

20. Итог по теме

- Функция нужна для того, чтобы не копировать один и тот же код снова и снова.
- Параметры - это входные данные функции, аргументы - конкретные значения при вызове.
- return возвращает значение наружу, а console.log() только показывает его в консоли.
- Стрелочная функция - это короткая запись функции.
- Массив - это список значений в одной переменной.
- Индекс первого элемента массива всегда равен 0.
- Чтобы уверенно работать с массивами, нужно сначала понять цикл for и свойство length.
- map() преобразует массив, filter() отбирает элементы, find() ищет первое совпадение, forEach() просто проходит по массиву.

Финальный совет

Не заучивайте методы массива как отдельные магические команды. Всегда задавайте себе вопрос: что я хочу получить на выходе? Новый массив, отфильтрованный массив, первый найденный элемент или просто действие для каждого элемента. Тогда выбор метода станет понятным.